1. Project introduction:

This project presents a classic Snake Game implemented on an Arduino Uno microcontroller, displayed on a 16x2 LCD screen using the I2C communication protocol. The game is controlled by four push buttons representing Up, Down, Left, and Right directions. The project demonstrates key concepts of embedded systems including GPIO handling, real-time game logic, custom LCD character rendering, and debounce-free button input using INPUT_PULLUP configuration.

The Snake Game is a well-known arcade game where the player controls a snake that grows longer each time it eats food. The goal is to eat as much food as possible without colliding with the snake's own body. This project brings that classic experience to a low-resource embedded platform.

2. Problem Statement (Definition):

Implementing an interactive real-time game on a microcontroller with very limited resources poses several challenges:

- The Arduino Uno has only 2KB of SRAM and 32KB of Flash memory, severely limiting the complexity of the game logic and display buffer.
- The 16x2 LCD provides only 32 character positions, requiring efficient use of custom characters to represent game elements (head, body, food).
- Button inputs must be read reliably without hardware debouncing circuits, using software-based INPUT_PULLUP mode instead.
- The game loop must run in real-time using non-blocking timing (millis()) to avoid freezing the input reading while the snake moves.

3. The Proposed solution:

The solution uses an Arduino Uno as the main controller connected to a 16x2 LCD via the I2C protocol (using only 2 data wires: SDA and SCL). Four push buttons are connected using INPUT_PULLUP mode, eliminating the need for external pull-down resistors.

The game logic is built around a coordinate array that tracks the snake's body positions. The snake moves at a configurable speed controlled by a non-blocking timer. Custom LCD characters are used to render the snake head, body segments, and food item with distinct visual shapes. The difficulty increases automatically as the player's score grows by reducing the movement delay.

3. Pros and Cons (advantage and disadvantage) :

Advantages:

- Low cost — uses only an Arduino Uno, LCD I2C module, and 4 buttons.
- Simple wiring — I2C reduces LCD connections to just 4 wires.
- Non-blocking game loop using millis() ensures smooth input response.
- Automatic difficulty scaling keeps the game engaging as score increases.
- INPUT_PULLUP eliminates the need for external pull-down resistors.

Disadvantages:

- Very limited display area (16x2 = 32 cells) restricts gameplay complexity.
- No persistent score storage — high scores are lost on power off.
- lcd.clear() on every frame causes slight visible flickering.

5.Tools And Software Used:

Hardware Components:

Component	Purpose
Arduino Uno	Main microcontroller
LCD 16x2 + I2C	Display with I2C module (PCF8574T, address 0x27)
Breadboard	For building the circuit
4x Push buttons (2-pin)	TO control the direction
Jumper Wires	For connections between components

Software:

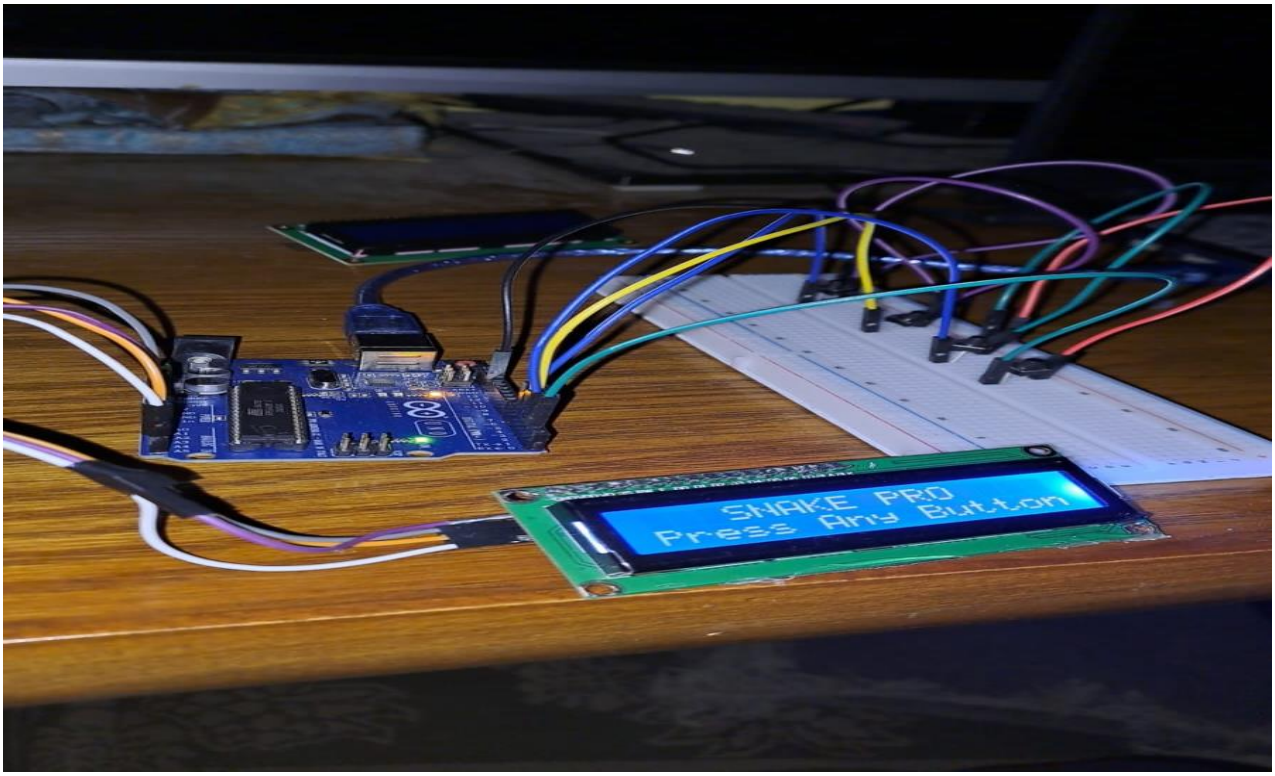
Software	Purpose
Arduino IDE	Writing and uploading the code
Liquid Crystal I2C library by Frank de Brabander	To control the LCD via I2C

6. Circuit design:

The following table summarizes all connections between components and the Arduino Uno:

Pin	Arduino Pin
GND	GND
VCC	5V
SDA	A4
SCL	A5
Putton(up)	D2,GND
Putton(down)	D3,GND
Putton(lift)	D4,GND
Putton(right)	D5.GND

7. final hardware Design:



8. Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const int btnUP = 2, btnDOWN = 3, btnLEFT = 4, btnRIGHT = 5;

byte head[] = {0x0E, 0x1F, 0x1F, 0x1F, 0x1F, 0x1F, 0x0E, 0x00};
byte body[] = {0x0E, 0x15, 0x1F, 0x1F, 0x1F, 0x15, 0x0E, 0x00};
byte food[] = {0x00, 0x0A, 0x04, 0x0E, 0x04, 0x0A, 0x00, 0x00};

int snakeX[20], snakeY[20];
int snakeLen, dirX, dirY, foodX, foodY, score;
```

```
unsigned long prevMillis = 0;
int gameDelay = 400;
bool running = false;

void setup() {
  lcd.init();
  lcd.backlight();
  lcd.createChar(0, body);
  lcd.createChar(1, head);
  lcd.createChar(2, food);

  pinMode(btnUP, INPUT_PULLUP);
  pinMode(btnDOWN, INPUT_PULLUP);
  pinMode(btnLEFT, INPUT_PULLUP);
  pinMode(btnRIGHT, INPUT_PULLUP);

  showStartScreen();
}

void showStartScreen() {
  lcd.clear();
  lcd.setCursor(3, 0); lcd.print("SNAKE PRO");
  lcd.setCursor(0, 1); lcd.print("Press Any Button");

while(digitalRead(btnUP)&&digitalRead(btnDOWN)&&digitalRead(btnLEF
T)&&digitalRead(btnRIGHT));
  initGame();
}

void initGame() {
  snakeLen = 3;
  score = 0;
  dirX = 1; dirY = 0;
  gameDelay = 400;
  for(int i=0; i<snakeLen; i++) { snakeX[i] = 3-i; snakeY[i] = 0; }
  spawnFood();
  running = true;
```

```
}

void spawnFood() {
  foodX = random(0, 16);
  foodY = random(0, 2);
}

void loop() {
  if (!running) return;


  if (digitalRead(btnUP) == LOW && dirY == 0)    { dirX = 0; dirY = -1; }
  if (digitalRead(btnDOWN) == LOW && dirY == 0)  { dirX = 0; dirY = 1; }
  if (digitalRead(btnLEFT) == LOW && dirX == 0)   { dirX = -1; dirY = 0; }
  if (digitalRead(btnRIGHT) == LOW && dirX == 0)  { dirX = 1; dirY = 0; }

  if (millis() - prevMillis >= gameDelay) {
    prevMillis = millis();
    moveSnake();
  }
}

void moveSnake() {
  int nextX = snakeX[0] + dirX;
  int nextY = snakeY[0] + dirY;

  if (nextX < 0) nextX = 15; else if (nextX > 15) nextX = 0;
  if (nextY < 0) nextY = 1; else if (nextY > 1) nextY = 0;

  for (int i = 0; i < snakeLen; i++) {
    if (nextX == snakeX[i] && nextY == snakeY[i]) { gameOver(); return; }
  }

  if (nextX == foodX && nextY == foodY) {
    score++;
    snakeLen++;
    if (gameDelay > 150) gameDelay -= 20;
```

```
    spawnFood();
  }

  for (int i = snakeLen - 1; i > 0; i--) {
    snakeX[i] = snakeX[i - 1];
    snakeY[i] = snakeY[i - 1];
  }
  snakeX[0] = nextX;
  snakeY[0] = nextY;

  draw();
}

void draw() {
  lcd.clear();
  lcd.setCursor(foodX, foodY); lcd.write(2)
  for (int i = 0; i < snakeLen; i++) {
    lcd.setCursor(snakeX[i], snakeY[i]);
    lcd.write(i == 0 ? 1 : 0);
  }
}

void gameOver() {
  running = false;
  lcd.clear();
  lcd.setCursor(3, 0); lcd.print("GAME OVER!");
  lcd.setCursor(4, 1); lcd.print("Score: "); lcd.print(score);
  delay(3000);
  showStartScreen();
}
```